

FPGA Implementation of CNN Accelerator with Pruning for ADAS Applications

Aimy Jose

Department of Electronics Engineering
Model Engineering College, Thrikkakara
APJ Abdul Kalam Technological University
Kerala, India
aimy jose.mec@gmail.com

Alense K.T.

Department of Electronics Engineering
College of Engineering, Poonjar
APJ Abdul Kalam Technological University
Kerala, India
alensethomas@gmail.com

Liya Gijo

Department of Electronics Engineering
College of Engineering, Poonjar
APJ Abdul Kalam Technological University
Kerala, India
liyagijo18@gmail.com

Jobymol Jacob

Department of Electronics Engineering
Model Engineering College, Thrikkakara
APJ Abdul Kalam Technological University
Kerala, India
joby@mec.ac.in

Abstract—Convolutional neural network plays a prominent role in computer vision applications such as advanced driver assistance systems which demand high accuracy and low latency. The need for low latency, resource limitation and power constraints are the major complexities that are associated with the deployment of the computationally intensive convolutional neural networks in ADAS applications. In this paper, an efficient CNN accelerator is implemented on Xilinx PYNQ-Z2 FPGA Board for traffic sign recognition. The CNN model is further accelerated by pruning the weights of the filters which has very little effect on accuracy. The experimental results show that the proposed method effectively achieves 5.65 times speed in terms of execution time with only 0.33 times drop in accuracy on LeNet-5 architecture.

Index Terms—FPGA accelerator, Convolutional Neural Networks, Pruning, Advanced Driver Assistance Systems, Traffic Sign Detection

I. INTRODUCTION

Convolutional neural network (CNN) plays a very crucial role in advanced driver assistance systems (ADAS) and in autonomous vehicles. CNN has wide range of applications especially in video surveillance, tracking, object detection and recognition, traffic monitoring etc. CNNs enable the system to make sense of the environment, detect objects of interest, and take fast and accurate decisions. They are essential in ADAS for tasks such as vehicle detection, pedestrian detection, lane detection, traffic sign recognition and many more. In the perspective of traffic safety, traffic sign recognition (TSR) is one among the major ADAS applications that enables the vehicle to recognize the road signs. By making use of this, the electronic control unit (ECU) can generate signals to control braking, steering and throttling action of the vehicle. However in case of time critical applications such as ADAS [1], the CNN architecture becomes more and more complex as the

number of layers increases and the number of computation operations also increases. The two important layers that mainly add to the bottlenecks during deployment are convolution layer and fully connected (FC) layer which are respectively computation-intensive and memory-intensive [2], [3]. This makes it challenging for the CNNs to be deployed in real time ADAS applications which have constraints with respect to response time, resource utilisation, memory requirements and power consumption. It is therefore essential to search for an efficient and optimised design for CNN accelerator [4].

The hardware platforms that are prominently used for CNN accelerators are graphical processing units (GPU), application specific integrated circuits (ASIC) and field programmable gate arrays (FPGA). The high reconfigurability, flexibility, integration capability and efficiency provided by FPGAs at optimal power consumption makes it well suited for accelerating CNN [5].

PYNQ FPGA allows for an efficient hardware and software co-design for deep learning applications. The use of PYNQ framework enables the designer to combine the benefits of reconfigurability of FPGA along with the use of python language and libraries for model development. [6] The programmable processor cores embedded with the FPGA further improves their performance advantages [7]. A. Maraoui et. al. [8] made use of PYNQ framework to implement a LeNet-5 based TSR application on PYNQ Z1 platform. They were able to achieve an acceleration of more than hundred times in terms of response time when comparing with the CPU and FPGA implementation. D. Rongshi et. al. [9] implemented the CNN accelerator model on ZYBO Z7 board and were able to achieve a data throughput of 0.343 GFLOP with a power consumption of 1.8 W in comparison with that of a general processor.

In this paper, a CNN accelerator with pruning technique

is implemented in FPGA for traffic sign recognition which overcomes the timing constraints. The Xilinx ZYNQ SoC based PYNQ-Z2 is used to implement the model.

In this work, a fully synthesizable LeNet-5 architecture Intellectual Property (IP) core has been designed in HLS and implemented in FPGA. The designed LeNet-5 core is then pruned and used to accelerate the CNN model for traffic sign recognition application with the dataset German traffic sign recognition benchmark (GTSRB) on the PYNQ-Z2 FPGA Board.

The paper is organized as follows. Section II discusses the underlying principles of traffic sign recognition using LeNet-5 model and its architecture. Section III deals with the design and FPGA implementation of LeNet-5 model for traffic sign recognition. In section IV, the experimental results are analyzed and performance evaluation is done. Section V concludes the work.

II. TRAFFIC SIGN RECOGNITION USING LENET-5

This section discusses the use of the CNN model LeNet-5, for traffic sign recognition application. During inference phase, the model is pruned for achieving better sparsity.

A. CNN for object detection

The most commonly used deep learning model for image processing and computer vision is CNN. It combines feature extraction and classification and can be used for wide variety of applications with highly accurate results. Different convolution kernels are used in the convolutional layers of CNN so as to obtain different types of features from the input data as required by the application. The size of the kernels in most of the cases are 3x3 or 5x5. Activation functions such as ReLU, leaky ReLU, sigmoid, tanh etc are used to introduce non linearity to the model [10]. The next important layer is the subsampling layer which is typically used to reduce the size of the feature map. This reduction is achieved by retaining majority of the dominant features lying within the region covered by the filter while discarding few of lesser ones. It reduces the computational complexity of the network due to the reduction of the spatial size of feature map. The most commonly used subsampling methods are max pooling and average pooling.

Depending upon the convolutional neural network architecture required for the application, multiple pairs of convolutional and subsampling layers are concatenated. After feature extraction, the output of the subsampling layer is flattened into one dimensional vector and passed onto a fully connected layer. This layer performs the classification by providing a label indicating the correct class that the input image belongs to.

B. Training of the LeNet-5 model

The model that is used in this work is LeNet-5 which has quite simple architecture compared to the other CNN models and hence, the training process is faster. In LeNet-5, as depicted in Fig. 1, the three sets of convolution layers and

two subsampling layers perform the task of feature extraction of the input data while the two fully connected layers perform classification [11]. Due to this, the number of parameters is very less compared to other models and is hence, suitable for implementation in hardware platforms. This final FC layer is succeeded by a softmax classifier. The tanh function is used as activation followed by average pooling. In order to reduce complexity, pruning can be done by removing all the weights of the filter channels that are close to zero and then reconnecting the layers so that the information flow still happens.

LeNet-5 is used for traffic sign recognition because of its faster training speed and the reduced number of parameters. The model is trained with the public dataset German traffic sign recognition benchmark (GTSRB) for traffic sign recognition application. This dataset includes 39,209 images that are classified into 43 categories. 80% of the images are used for training and 20% for testing the model. Initially, the images of the dataset are pre-processed and the image size is converted to 32x32 pixels. One-hot encoding is then done to provide the dataset labels with a categorical representation as a binary vector. During each iteration, the network weights get updated and the gradient error between the labels and the predictions are minimized using adaptive moment estimation (ADAM) optimizer. The batch size is set at 32 while the learning rate is kept at 0.001. The weight updation continues until the accuracy reaches a maximum value and the loss converges to a minimum value. The performance metrics of the model are estimated at this stage. Fig. 2. depicts the training and validation accuracy and loss. The training accuracy is observed as 92.3 % while the loss percentage is 0.2% . The val_acc represents the accuracy obtained during validation while the val_loss represents the loss occurred during validation which are 92.3% and 0.6% respectively.

During testing phase the remaining 20% of the images are used to classify the images into respective categories using the trained model. After training all the parameters including weights of the layers of the model, the biases are saved as .npy files.

C. Heuristic pruning of the existing architecture for better sparsity

Weights and biases obtained after training the model represents the trainable/learnable parameters. From Table I, it can be observed that the total number of trainable parameters of the LeNet-5 model for the Traffic sign recognition application dealing with RGB image is 1,34,155. The number of parameters increases even further as the architecture becomes more and more complex. This can be of concern when deploying a model in real-time applications. The use of pruning eliminates the redundant parameters from the neural network architecture that were obtained after training process. In addition, it helps in minimizing the resource utilization during the inference phase and decreases the response time [12], [13]. Pruning can be performed on the weights on a layer by layer basis followed by iterative retraining in order to compensate for the lost accuracy

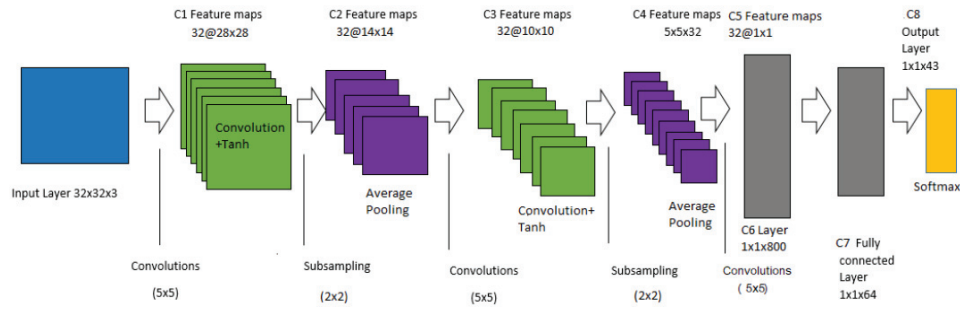


Fig. 1. Network architecture of LeNet-5

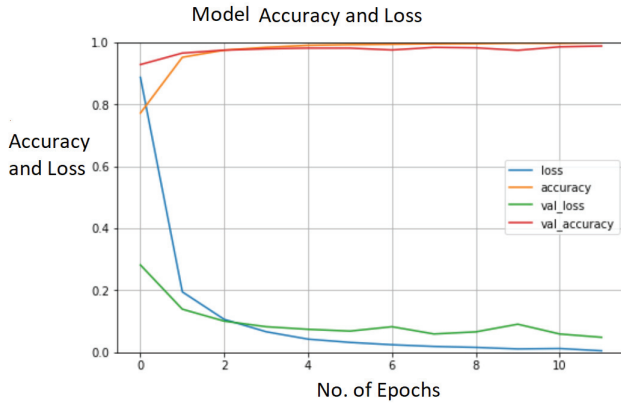


Fig. 2. Training and validation accuracy and loss of the model

[14]. However, this process is time consuming and can result in additional pruning of layers especially in case of complex networks. In order to avoid this, independent pruning is done where all the layers are pruned consecutively [15]. Sparse pruning is done in which individual weights with the lowest magnitudes are removed globally without considering the layer associated [16].

TABLE I
LENET-5 ARCHITECTURE SUMMARY

Layer	Size	Kernel size	Stride	Filters	Trainable Parameters
Input (Image)	32x32	-	-	-	-
Convolution C1	28x28	5x5	1	32	2432
Average Pooling C2	14x14	2x2	2	32	0
Convolution C3	10x10	5x5	1	32	25632
Average Pooling C4	5x5	2x2	2	32	0
Convolution C5	1x1	5x5	1	32	25632
Dense Layer C6	800	-	-	-	26400
Dense Layer C7	64	-	-	-	51264
Output Layer (Fully Connected) C8	43	-	-	-	2795
					134,155

III. DESIGN AND IMPLEMENTATION OF CNN ACCELERATOR IN FPGA

In this section, the design of the LeNet-5 model for traffic sign recognition is done. The designed IP core is packaged as a python overlay on PYNQ board. Further, the model for traffic sign recognition is implemented on FPGA.

A. Design of LeNet-5 model in HLS for FPGA Implementation

In this section, the design of LeNet-5 IP core for traffic sign recognition targeting the Xilinx FPGA are discussed. Xilinx high level synthesis (HLS) tool has been used for design and the synthesized design was implemented on PYNQ Z2 FPGA board. The input to the LeNet-5 is a 32x32 pixel RGB image. The kernel size is fixed at 3x3. After training the model using Python, the .npy file that contains the model parameters such as weights and biases are exported and stored in a header file to be used during the full model replication in HLS. The LeNet-5 model which consists of two convolution layers, two subsampling layers, a flattening layer and two fully connected layers is designed in HLS. The use of HLS reduces the hardware development time and makes the process less complex. The creation of IPs using Vivado HLS is done using C, C++ and System C and the codes are converted directly to RTL. The model was optimized by making use of the PRAGMAS for higher execution performance and data throughput.

B. Packaging of the IP Core as a python overlay on PYNQ Z2 board

The LeNet-5 model is then synthesized and custom user IP Core is generated using Xilinx Vivado IP integrator. The LeNet-5 IP is then added to the Vivado repository and stored in the IP catalogue. The LeNet-5 IP is then connected with the necessary IP cores for hardware implementation on Pynq Z2 device. Fig.3. represents the block design of the entire system. AXI interface was employed to carry out the communication between LeNet-5 IP and the ZYNQ processing system. Processor system reset block ensures that the entire system is under the control of a common reset. Clock wizard manages the clocking requirements of the system. Once the block design is completed, the bitstream is generated and is exported as a python overlay that can be loaded on to a PYNQ

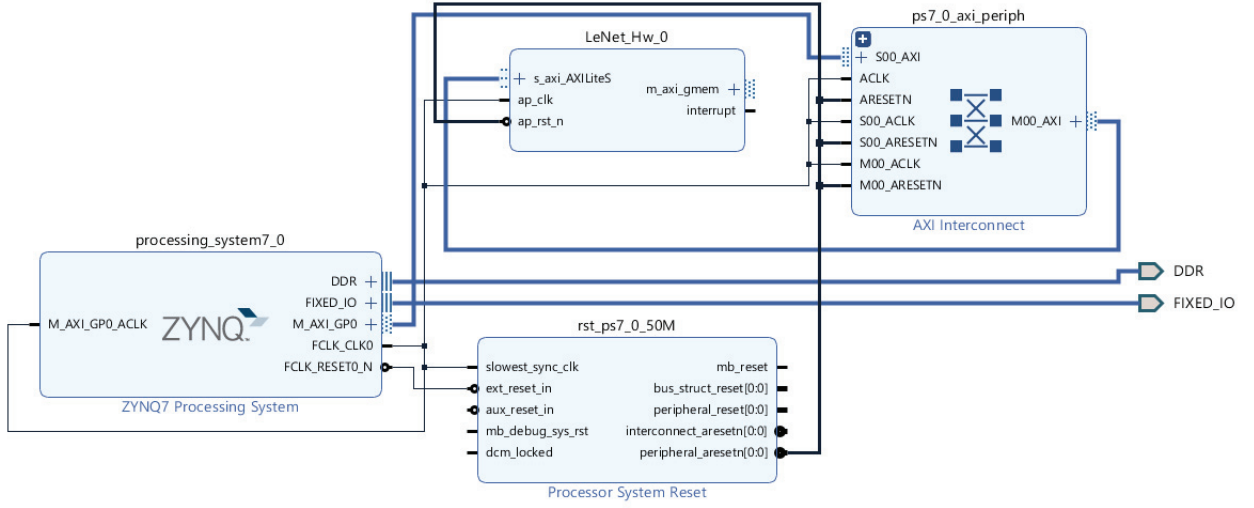


Fig. 3. Block design of the model

development board. The exported python overlay consists of two parts. The bitstream file (.bit) contains all the configuration information from the hardware design and the blocks that are essential to define the internal logic. It also contains the device specific information from the files that are associated with the FPGA target. Next is the Tcl file (.tcl) from the Vivado IP Integrator block design for the Programmable Logic (PL) design which is used by PYNQ to automatically identify the ZYNQ system configuration, interrupts, resets and other control signals.

A hardware handoff (.hwh) file was also loaded along with the .bit and .tcl file to the PYNQ device. It is used by the software tool to abstract all the informations such as buses, ports and pins used in systems, which are needed to build a target application on PYNQ. Workflow of the same is shown in Fig. 4.

C. Implementation of LeNet-5 model in PYNQ Board

The exported overlay of the designed LeNet-5 model is then used for traffic sign recognition with the dataset GTSRB in Jupyter Notebook environment, once the PYNQ Z2 device is setup. The forward inference of the LeNet-5 model implemented on the PYNQ FPGA device is replicated on the Jupyter Notebook environment and is used to test the model by inputting the image. The model was then tested to carry out the image recognition by exploiting the hardware-software co-design approach. Pruning of the weights were done at this stage to further reduce the computational complexity of the model. During this phase, the execution time for processing input image is analyzed.

IV. RESULT ANALYSIS

In this section, the experimental results obtained are analyzed and the performance of the proposed model is evaluated

Dataset German Traffic Sign Recognition Benchmark (GTSRB)

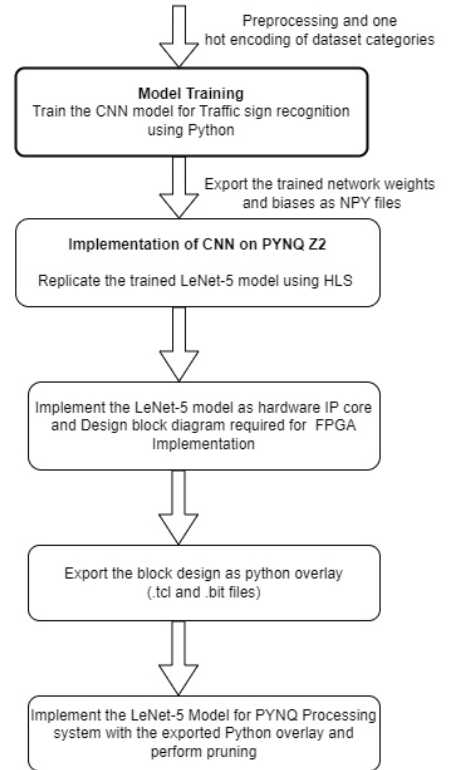


Fig. 4. Design workflow

in terms of the metrics such as efficiency, resource utilization and latency.

Fig. 5. represents the classification result obtained during software validation. The time taken for the system to classify the image is 0.133 seconds.

```

start = time.time()
pred = model.predict_classes(img)[0]
print(pred)
end = time.time()
sign = classes[pred]
print(sign)
print("Time taken:",round(end-start,5), "seconds")

```

```

11
Right-of-way at intersection
Time taken: 0.13353 seconds

```

Fig. 5. Classification result observed during validation



```

12
Priority road

```



```

34
Turn left ahead
100 images time: 4.7 secs

```

Fig. 6. Prediction result obtained before network pruning with execution time for 100 images obtained as 4.7 seconds

Fig. 6. represents the prediction result as observed in Jupyter Notebook environment before pruning is done on the model. The inference result of priority road and turn left ahead can be observed along with the time taken for the prediction of 100 images. In Fig. 7. the inference result of the keep right sign is represented. Fig. 8. represents the experimental setup of the implementation with the classified 'NO Entry' sign on the screen.



```

38
Keep right
0.02 secs

```

Fig. 7. Prediction result after pruning the network

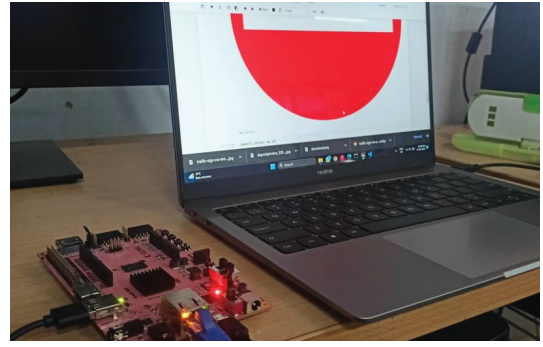


Fig. 8. Experimental setup of the implementation

Table II depicts the synthesis result of the LeNet-5 model. It represents the resources utilized, when the model is implemented in PYNQ Z2 FPGA. It shows that the proposed prototype occupies 44% of LUTs, 62% of LUTRAMs, 24% of FFs, 59% of BRAMs and 9% of DSP.

TABLE II
SYNTHESIS REPORT OF THE LeNET-5 MODEL

Resource	Available	Resource Utilization	Resource Utilization %
Slice LUTs	53200	23756	44
LUT RAMs	280	175	62
Flipflops	106400	25906	24
DSP Slices	220	20	9

The execution time of the model during software implementation for processing a single image is observed as 0.133 seconds. The execution time further decreased to 0.047 seconds after the model is implemented in FPGA which further decreased to 0.02 seconds after the accelerated model was pruned. The accuracy of the model which was 92.3% during software implementation decreased to 92% after pruning.

V. CONCLUSION

In this paper, a CNN accelerator optimized with pruning is implemented in FPGA for traffic sign recognition system. The model is based on LeNet-5 network architecture. It was first trained using Python and the corresponding parameters were exported. The model was replicated and synthesized using Xilinx HLS tool while making use of the imported parameters from the training process. Block design was done and the python overlay was imported to Jupyter Notebook for observing the acceleration achieved on PYNQ Z2. Experimental results show that the proposed method has achieved considerable improvement in execution time with only a minimal decrease in accuracy on LeNet-5 architecture.

ACKNOWLEDGMENT

The authors would like to acknowledge Ministry of Minority Affairs (MoMA) for providing the financial support through Maulana Azad National Fellowship Scheme (MANF) to carry

out this research work. The authors would also like to acknowledge DST for rendering infrastructural support through FIST project.

REFERENCES

- [1] X. Wang, K. Huang and A. Knoll, "Performance Optimisation of Parallelized ADAS Applications in FPGA-GPU Heterogeneous Systems: A Case Study With Lane Detection," in *IEEE Transactions on Intelligent Vehicles*, vol. 4, no. 4, pp. 519-531, Dec. 2019, doi: 10.1109/TIV.2019.2938092.
- [2] J. Kim, J. -K. Kang and Y. Kim, "A Low-Cost Fully Integer-Based CNN Accelerator on FPGA for Real-Time Traffic Sign Recognition," in *IEEE Access*, vol. 10, pp. 84626-84634, 2022, doi: 10.1109/ACCESS.2022.3197906.
- [3] T. Abtahi, C. Shea, A. Kulkarni and T. Mohsenin, "Accelerating Convolutional Neural Network With FFT on Embedded Hardware," in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 9, pp. 1737-1749, Sept. 2018, doi: 10.1109/TVLSI.2018.2825145.
- [4] T. V. Huynh, "FPGA-based Acceleration for Convolutional Neural Networks on PYNQ-Z2," *International Journal of Computing and Digital Systems*, vol. 11, no. 1, pp. 441-449, Jan. 2022, doi: <https://doi.org/10.12785/ijcds/110136>.
- [5] M. Haileseilasie, S. R. Hasan, F. Khalid, F. A. Wad and M. Shafique, "FPGA-Based Convolutional Neural Network Architecture with Reduced Parameter Requirements," 2018 IEEE International Symposium on Circuits and Systems (ISCAS), Florence, Italy, 2018, pp. 1-5, doi: 10.1109/ISCAS.2018.8351283.
- [6] J. Wang and L. Xu, "PYNQ Framework Based Object Recognition Implementation Using Convolution Neural Network (CNN)," *Review of Computer Engineering Studies*, vol. 9, no. 4, pp. 136-140, Dec. 2022, doi: <https://doi.org/10.18280/rces.090402>.
- [7] M. Dhouibi, A. K. Ben Salem and S. B. Saoud, "CNN for object recognition implementation on FPGA using PYNQ framework," 2020 IEEE Eighth International Conference on Communications and Networking (ComNet), Hammamet, Tunisia, 2020, pp. 1-6, doi: 10.1109/ComNet47917.2020.9306094.
- [8] A. Maraoui, S. Messaoud, S. Bouaafia, A. C. Ammari, L. Khriji and M. Machhout, "PYNQ FPGA Hardware implementation of LeNet-5-Based Traffic Sign Recognition Application," 2021 18th International Multi-Conference on Systems, Signals & Devices (SSD), Monastir, Tunisia, 2021, pp. 1004-1009, doi: 10.1109/SSD52085.2021.9429480.
- [9] D. Rongshi and T. Yongming, "Accelerator Implementation of Lenet-5 Convolution Neural Network Based on FPGA with HLS," 2019 3rd International Conference on Circuits, System and Simulation (ICCSS), Nanjing, China, 2019, pp. 64-67, doi: 10.1109/CIRSYS-SIM.2019.8935599.
- [10] A. M. Salman, A. S. Tulan, R. Y. Mohamed, M. H. Zakhari and H. Mostafa, "Comparative Study of Hardware Accelerated Convolution Neural Network on PYNQ Board," 2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES), Giza, Egypt, 2020, pp. 578-582, doi: 10.1109/NILES50944.2020.9257899.
- [11] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998, doi: <https://doi.org/10.1109/5.726791>.
- [12] Sui, Xuefu, Qunbo Lv, Liangjie Zhi, Baoyu Zhu, Yuanbo Yang, Yu Zhang, and Zheng Tan. 2023. "A Hardware-Friendly High-Precision CNN Pruning Method and Its FPGA Implementation" *Sensors* 23, no. 2: 824. <https://doi.org/10.3390/s23020824>.
- [13] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. Graf, Published as a conference paper at ICLR 2017, "Pruning Filters for Efficient ConvNets." Available: <https://arxiv.org/pdf/1608.08710.pdf>.
- [14] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning Convolutional Neural Networks for Resource Efficient Inference," arXiv:1611.06440 [cs, stat], Jun. 2017, Available: <https://arxiv.org/abs/1611.06440>.
- [15] L. Liebenwein, C. Baykal, B. Carter, D. Gifford, and D. Rus, "Lost in Pruning: The Effects of Pruning Neural Networks beyond Test Accuracy," arXiv:2103.03014 [cs], Mar. 2021, Available: <https://arxiv.org/abs/2103.03014>.
- [16] M. Pietron and M. Wielgosz, "Retrain or not retrain? – efficient pruning methods of deep CNN networks," arXiv.org, Feb. 12, 2020. <https://arxiv.org/abs/2002.07051>.